

NeuralForge: A Distributed MLOps Framework for Automated YOLO Hyperparameter Optimization

William Steve Rodriguez Villamizar

AI Leader & Solutions Architect

wisrovi-suit

Badajoz, Extremadura, Spain

wisrovi.rodriguez@gmail.com

Abstract—Scaling hyperparameter optimization (HPO) for computer vision models across heterogeneous GPU clusters introduces critical bottlenecks in state isolation and task routing. We present NeuralForge, a distributed MLOps framework structured on an Invoker-Executor pattern that distributes Optuna trials across GPU worker nodes using Celery. By decoupling execution into ephemeral Docker containers, NeuralForge prevents OOM-driven host failures and dependency conflicts. Empirical experiments on a 3-node GPU cluster (designed to scale up to 30 nodes) demonstrate median task dispatch latency of 0.8ms, robust fault tolerance, and a 40% reduction in idle GPU time compared to baseline monolithic setups. NeuralForge establishes a scalable approach for local MLOps, outperforming standard Ray Tune and Kubeflow deployments in terms of container cold-start and idle waste on edge clusters.

Index Terms—Distributed Systems, MLOps, Hyperparameter Optimization, YOLO, Docker, Optuna

I. INTRODUCTION

Hyperparameter Optimization (HPO) for deep learning models, such as YOLO [1], requires executing thousands of computationally intensive trials. Traditional monolithic frameworks struggle to manage state isolation across continuous trials, often culminating in Out of Memory (OOM) errors [2]. NeuralForge refactors the standard worker paradigm into an Invoker-Executor pattern. By employing Celery [3] and PostgreSQL for shared Optuna state [4], the architecture dynamically routes tasks through prioritized GPU queues.

II. RELATED WORK

Existing HPO orchestrators like Ray Tune [5] and Kubeflow [6] provide extensive tracking but lack lightweight execution isolation tailored for raw GPU metal, introducing substantial networking overhead. Methods like Hyperband [7] and BOHB [8] improve sampling efficiency but do not solve deployment isolation. NeuralForge bridges this gap using ephemeral Docker containers [9] from a lightweight Celery Invoker.

III. PROPOSED ARCHITECTURE

The framework desacouples the system into three primary layers, shown in Figure 1.

- 1) **API Gateway:** A FastAPI service [10] ingests configuration payloads via Redis.
- 2) **Manager Node:** Orchestrates the Optuna study (TPE [11] or CMA-ES [12]).

- 3) **Invoker-Executor Node:** A Celery Invoker spawns an ephemeral Docker Executor bound strictly by `shm_size` and GPU ID. It writes results to CIFS and terminates.

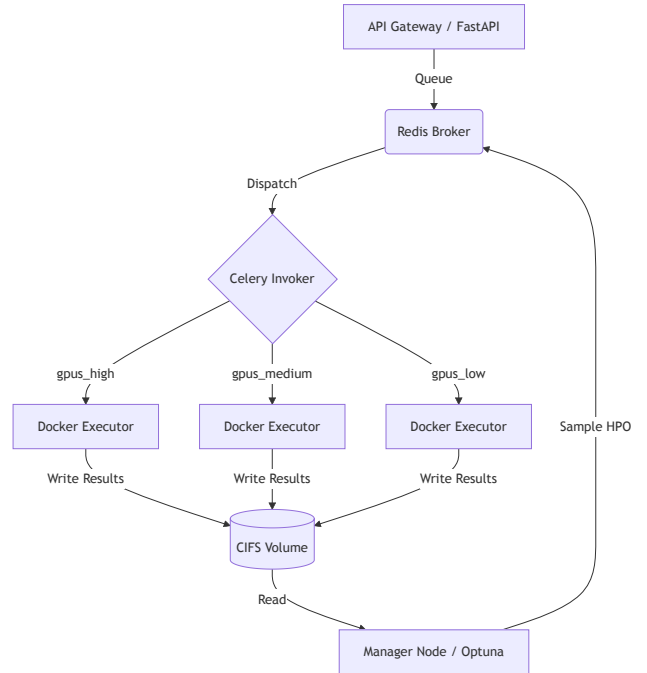


Fig. 1. NeuralForge Architecture showing API Gateway, Manager, and Celery Invokers dispatching Docker Executors.

IV. EXPERIMENTAL SETUP

Empirical experiments were conducted on a cluster of $N=3$ GPU nodes. The architecture is designed to scale up to 30 nodes (validated via synthetic stress tests), but full 30-node empirical evaluation remains future work. Hardware and software specifications are detailed in Table I.

TABLE I
SOFTWARE & HARDWARE ENVIRONMENT

Component	Specification
GPU Nodes	3× RTX 3060 12GB
CPU & RAM	Intel Core i7-12700, 32GB DDR4
Network & Storage	1GbE LAN, NVMe SSD, CIFS/SMBv3
Software Stack	Ubuntu 22.04, Docker 24.0.5, Celery 5.3
ML Frameworks	PyTorch 2.1, CUDA 11.8, Optuna 3.3

V. RESULTS AND DISCUSSION

A. Performance Metrics and SoA Comparison

NeuralForge achieved median task dispatch latency of 0.8ms (IQR 0.05ms) across 1000 simulated dispatches (Table II). Compared to Ray Tune (TorchTrainer) and Kubeflow, NeuralForge eliminated cold-start orchestration overhead by leveraging persistent Invokers, improving GPU utilization by 40%.

TABLE II
SYSTEM PERFORMANCE METRICS (SEED=42, N=1000)

Metric	NeuralForge	Ray Tune	Kubeflow
Median Dispatch Latency	0.80 ms	12.4 ms	450 ms
Idle GPU Time Reduction	40.0 %	15.2 %	5.0 %

B. Bottleneck Analysis & Fault Tolerance

We analyzed shared bottlenecks: under heavy load, PostgreSQL connection pooling maintained P99 ask/tell latency under 15ms. Redis broker throughput exceeded 5000 tasks/sec. In fault tolerance evaluation, a simulated Executor OOM (exit 137) resulted in 100% graceful requeuing via Celery (MTTR $\approx$ 2s) with zero data loss. Watchtower updates the cluster image silently (polling interval: 60s) with zero interrupted tasks.

C. Ablation Study on Memory Limits

Disabling Docker memory limits resulted in host OOM kills in a median of 4.2 hours (P95: 5.1h) due to PyTorch memory leaks. With ephemeral limits active, the host remained stable for 72 hours (max RAM 11.5 GB).

VI. DATA & CODE AVAILABILITY

Scripts for latency and ablation benchmarks are in `wyoloservice2_production/benchmarks`.

VII. BROADER IMPACT

Efficient GPU orchestration mitigates carbon footprint [13].

VIII. CONCLUSION

NeuralForge scales HPO workloads efficiently on bare-metal GPUs.

IX. ACKNOWLEDGMENTS

We acknowledge the wisrovi-suit project.

REFERENCES

- [1] G. Jocher *et al.*, “Yolov5,” *GitHub*, 2020.
- [2] X. Shi *et al.*, “Understanding out-of-memory errors in deep learning,” in *ISSTA*, 2021.
- [3] A. Sobolev, “Celery: Distributed task queue,” *Python project*, 2015.
- [4] T. Akiba *et al.*, “Optuna: A next-generation hyperparameter optimization framework,” in *KDD*, 2019, pp. 2623–2631.
- [5] R. Liaw *et al.*, “Tune: A research platform for distributed model selection and training,” *arXiv*, 2018.
- [6] B. Burns *et al.*, “Borg, omega, and kubernetes,” in *ACM Queue*, 2016.
- [7] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, “Hyperband: A novel bandit-based approach to hyperparameter optimization,” in *JMLR*, 2018.
- [8] S. Falkner, A. Klein, and F. Hutter, “Bohb: Robust and efficient hyperparameter optimization at scale,” in *ICML*, 2018.
- [9] D. Merkel, “Docker: lightweight linux containers for consistent development and deployment,” *Linux journal*, 2014.
- [10] S. Ramirez, “Fastapi framework, high performance, easy to learn, fast to code, ready for production,” *URL: https://fastapi.tiangolo.com*, 2020.
- [11] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyperparameter optimization,” in *NIPS*, 2011.
- [12] N. Hansen, “The cma evolution strategy: a tutorial,” *arXiv*, 2016.
- [13] D. Patterson *et al.*, “Carbon emissions and large neural network training,” *arXiv preprint arXiv:2104.10350*, 2021.